

Nucleus AI — Phase 1 Audit Report

Date: June 3, 2026
Scope: Full-stack audit of the Nucleus AI platform (Next.js frontend + FastAPI backend) prior to the Phase 1 production-readiness overhaul.

1. Existing Features Inventory

1.1 Frontend Pages (Next.js App Router — frontend/src/app)

Route	Purpose	Status
/	Root — redirects to /dash-board or /login based on auth	✓ Working
/login	Credentials login (NextAuth)	✓ Working
/signup	Self-service registration	✓ Working
/dashboard	Main dashboard landing	✓ Working
/brand-profile	Manage brand profiles	✓ Working
/campaign-generator	AI campaign content generation (streaming)	✓ Working
/campaigns	Campaign list (frontend-only, no DB model pre-Phase 1)	⚠ Partial
/context-vault	Document upload / knowledge base	✓ Working
/analytics	Attribution / analytics charts	✓ Working
/settings	User settings	⚠ Partial

Shared UI: components/dashboard/* (sidebar, header, layout, charts) and a shadcn/ui component library under components/ui/* .

1.2 Backend API Endpoints (FastAPI — backend/api/v1)

Endpoint	Module	Purpose
GET /health , GET /api/v1/health	health	Liveness checks
POST /api/v1/context/ingest	context	Ingest text/PDF → chunk → embed → Qdrant
POST /api/v1/context/search	context	Semantic search with brand/content filters
POST /api/v1/workflow/execute	workflow	Run multi-agent LangGraph workflow (sync)
POST /api/v1/workflow/execute/async	workflow	Queue workflow (background task)
GET /api/v1/workflow/status/{job_id}	workflow	Poll async job status
POST/GET/PUT/DELETE /api/v1/campaigns	attribution	Campaign CRUD
POST/GET /api/v1/spend...	attribution	Spend tracking + summaries
POST/GET /api/v1/conversions...	attribution	Conversion events + summaries
GET /api/v1/attribution/{roas,cac,ltv,dashboard,report}	attribution	Attribution metrics

1.3 Frontend API Routes (frontend/src/app/api)

Route	Method	Purpose
/api/auth/[...nextauth]	*	NextAuth handler
/api/auth/login	POST	Redundant manual login (unused by NextAuth)
/api/signup	POST	Register user
/api/brands	GET/POST	List / create brand profiles
/api/brands/[id]	PUT/DELETE	Update / delete brand
/api/documents	GET	List uploaded documents
/api/upload/presigned	POST	Generate S3 pre-signed upload URL
/api/upload/complete	POST	Persist upload record
/api/workflow/execute	POST	Proxy to LLM for content generation (streaming)
/api/health	GET	Frontend health

1.4 Authentication System (Pre-Phase 1)

- **NextAuth.js v4** with a **Credentials provider** (email + password).
- Passwords hashed with **bcryptjs** (cost 12).
- **JWT session strategy**; session exposed `id`, `email`, `name`.
- Edge **middleware** (`src/middleware.ts`) protecting all non-public routes and redirecting based on auth state.

1.5 Database Models (Pre-Phase 1)

Frontend (Prisma):

- `User` — `id`, `name`, `email`, `password`, `image`, `timestamps`.
- `BrandProfile` — `name`, `industry`, `targetAudience`, `brandVoice`, `description`, `logoUrl`, `userId`.
- `UploadedDocument` — `fileName`, `fileType`, `cloudStoragePath`, `isPublic`, `brandId`, `status`, `userId`.

Backend (SQLAlchemy, separate concern):

- `Campaign`, `SpendLog`, `ConversionEvent` (used by the Attribution Engine; not shared with Prisma).

2. Missing Features Identified

2.1 Authentication Gaps

- **✗** No **email verification** flow (no token field, no verify endpoint).
- **✗** No **password reset** (no reset token / expiry, no endpoints).

- **✗** No **account lockout** / brute-force protection.
- **✗** No **last-login tracking**.
- **✗** No re-validation of account **status** (suspended users could log in).

2.2 RBAC

- **✗** No **roles** on the `User` model — every user had identical capabilities.
- **✗** No permission matrix, no role-based route protection, no UI gating.

2.3 Admin Panel

- **✗** No admin pages: user management, role assignment, audit log viewer, system settings.

2.4 User / Team Management

- **✗** No way to invite, list, suspend, or change roles of users.
- **✗** No ownership/creator tracking on brands or campaigns.

2.5 Permission Enforcement

- **✗** API routes only checked “is logged in”, never “is allowed”.
- **✗** No multi-tenant data scoping beyond `userId` equality on a couple of routes.

2.6 Data Model Gaps

- **✗** No **Campaign** model in Prisma (frontend campaigns page had no persistence).
- **✗** No first-class **Document** model for the Context Vault (status/embedding tracking).
- **✗** No **AuditLog** model.
- **✗** No **Notification** model.

3. Broken / Incomplete Features

Area	Issue
<code>/campaigns</code> page	No backing Prisma model → no real persistence in the frontend layer.
<code>/api/auth/login</code>	Duplicate of NextAuth credential flow; returns a user object but does not establish a session — dead/confusing code.
<code>/settings</code> page	Limited functionality; no profile/password update wired to a secured endpoint.
Document records	<code>UploadedDocument.status</code> is a free-form string; no processing/embedding state machine.
Backend vs frontend models	<code>Campaign</code> exists only in SQLAlchemy; the frontend had no equivalent, causing a data-model split.

4. Security Issues

Severity	Issue	Status after Phase 1 Part 1
High	No RBAC — any authenticated user could call any endpoint	✓ Fixed (permission guards)
High	Suspended accounts could still authenticate	✓ Fixed (status check in <code>authorize</code>)
High	No brute-force protection on login	✓ Fixed (lockout after 5 attempts / 15 min)
Medium	Weak/!no input validation on signup (no email format / min length)	✓ Fixed
Medium	No audit trail for sensitive actions	✓ Added (<code>AuditLog</code> + <code>recordAudit</code>)
Medium	Email not normalized (case-sensitive duplicates possible)	✓ Fixed (lowercased)
Medium	No rate limiting at the app layer	⚠ Handled at Nginx (prod); app-layer limiter deferred
Low	No CSRF tokens on custom POST routes	⚠ NextAuth covers auth; deferred for custom mutations
Low	Secrets present in <code>frontend/.env</code> (hosted DB creds)	📄 Gitignored; documented in <code>ENV_SETUP.md</code>

Note: Rate limiting is enforced at the Nginx reverse proxy in production (`nginx/conf.d/default.conf`). App-level rate limiting and CSRF for custom mutating routes are recommended for a later Phase 1 part.

5. Code Quality Issues

Issue	Notes	Status
Dead code: <code>/api/auth/login</code>	Not used by NextAuth flow	Flagged (kept for now; recommend removal)
Inconsistent auth checks	Each route re-implemented <code>getSession</code> + 401	✓ Centralized via <code>middleware/rbac.ts</code>
No shared permission logic	—	✓ Added <code>lib/permissions.ts</code>
Loose typing on session	<code>any</code> casts in callbacks	✓ Typed via <code>next-auth.d.ts</code> augmentation
Free-form status strings	<code>UploadedDocument.status</code>	New <code>Document</code> model uses enums
Missing validation	signup/brand create	✓ Added basic validation
Model split (Prisma vs SQLAlchemy)	Campaign duplication risk	Documented; Prisma <code>Campaign</code> added for frontend persistence

6. Phase 1 Part 1 — Changes Delivered

- Database schema overhaul** (`frontend/prisma/schema.prisma`):
 - `Role`, `UserStatus`, `CampaignStatus`, `DocumentType`, `ProcessingStatus`, `NotificationType` enums.
 - Enhanced `User` (role, status, email verification, password reset, login tracking, logout).
 - Enhanced `BrandProfile` (logo, website, brandColors JSON, guidelines, createdBy).
 - New `Campaign`, `Document`, `AuditLog`, `Notification` models.
 - Migrations: `0_init` (baseline) + `*_phase1_rbac_overhaul` (additive — **no data loss**).
- RBAC foundation:**
 - `lib/permissions.ts` — roles, granular permissions, permission matrix, helpers.
 - `middleware/rbac.ts` — server-side route guards (`requireAuth`, `requirePermission`, `requireRole`, ...).
 - `hooks/usePermissions.ts` — client-side UI gating.
 - `lib/audit.ts` — best-effort audit logging.
- Auth hardening:** role/status in JWT + session, logout, last-login, suspended-account block.
- API route protection:** brands, documents, uploads, workflow, signup now permission-checked & audited.
- Seed script:** default **Super Admin** + Admin/Editor/Viewer demo accounts.
- Docs:** this report + `PERMISSION_MATRIX.md`.

7. Recommendations for Subsequent Phase 1 Parts

1. Build the **admin panel** UI (user management, role assignment, audit viewer, notifications).
2. Implement **email verification** and **password reset** endpoints + emails (fields already in schema).
3. Add **app-layer rate limiting** and **CSRF** protection for custom mutating routes.
4. Add a Prisma-backed **Campaign** management UI and migrate the `/campaigns` page to it.
5. Wire the new **Document** model into the Context Vault ingest pipeline (processing/embedding status).
6. Remove the dead `/api/auth/login` route.
7. Add automated tests for RBAC guards and auth flows.